

Assignment #A: dp & bfs

Updated GMT+8 Nov 29, 2024

2024 fall, Compiled by 徐至晟, 光华

说明:

- 1) 请把每个题目解题思路 (可选), 源码Python, 或者C++ (已经在Codeforces/Openjudge上AC), 截图 (包含Accepted), 填写到下面作业模版中 (推荐使用 typora <https://typoraio.cn>, 或者用 word)。AC 或者没有AC, 都请标上每个题目大致花费时间。
- 2) 提交时候先提交pdf文件, 再把md或者doc文件上传到右侧“作业评论”。Canvas需要有同学清晰头像、提交文件有pdf、“作业评论”区有上传的md或者doc附件。
- 3) 如果不能在截止前提交作业, 请写明原因。

1. 题目

LuoguP1255 数楼梯

dp, bfs, <https://www.luogu.com.cn/problem/P1255>

思路:

递推; 矩阵快速幂: n 的二进制第 k 位是 1 (右数第一个是第 0 位) 则答案矩阵乘递推矩阵的 2^k 次方。时间复杂度 $\Theta(\log n)$, 不过本题 $\Theta(n)$ 的普通递推即可通过。

Python 自带高精度好评

$$a_{n+1} = a_n + a_{n-1}$$

$$\begin{pmatrix} a_{n+1} & a_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} a_n & a_{n-1} \end{pmatrix}$$

代码:

```
n=int(input())
a=[[1,1],[1,0]]
f=[1,1]
g=[0,0]
while n:
    if n%2:
        g[0]=f[0]*a[0][0]+f[1]*a[1][0]
        g[1]=f[0]*a[1][0]+f[1]*a[1][1]
        f[0]=g[0]
        f[1]=g[1]
    b=[[0 for _ in range(2)] for _ in range(2)]
    for i in range(2):
        for j in range(2):
            for k in range(2):
                b[i][j]+=a[i][k]*a[k][j]
    for i in range(2):
        for j in range(2):
```

```
a[i][j]=b[i][j]
n//=2
print(f[1])
```

代码运行截图 (至少包含有"Accepted")

soy_un_perro
11-29 14:22:34

Accepted
100

P1255 数楼梯

洛谷
158ms / 3.79MB / 472B Pyt

27528: 跳台阶

dp, <http://cs101.openjudge.cn/practice/27528/>

思路:

答案为 2^{n-1}

代码:

```
n=int(input())
print((1<<(n-1)))
```

代码运行截图 (至少包含有"Accepted")

24n2400015881(David_X)

27528: 跳台阶

Accepted

3904kB 31ms

33 B

Python3

刚刚

474D. Flowers

dp, <https://codeforces.com/problemset/problem/474/D>

思路:

简单的递推。每次最后吃的花是1红或k白。需要使用前缀和数组

代码:

```
t,k=map(int,input().split())
f=[1]
s=[1]
for i in range(1,100001):
    f.append(f[i-1])
    if i>=k:
        f[i]=(f[i]+f[i-k])%1000000007
    s.append((s[i-1]+f[i])%1000000007)
while t:
    a,b=map(int,input().split())
    print((s[b]-s[a-1])%1000000007)
    t-=1
```

代码运行截图 (至少包含有"Accepted")

LeetCode5.最长回文子串

dp, two pointers, string, <https://leetcode.cn/problems/longest-palindromic-substring/>

思路:

采用 $\Theta(n^2)$ 的枚举算法, 枚举子串的对称中心, 向两边逐个对比, 子串长度为奇偶分别操作。

没有使用Manacher算法

代码:

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n=len(s)
        L=1
        Ans=s[0:1]
        for piv in range(n):
            i=piv
            j=piv
            cnt=1
            while i>0 and j<n-1:
                if s[i-1]==s[j+1]:
                    cnt+=2
                    i-=1
                    j+=1
                else:
                    break
            if L<cnt:
                L=cnt
                Ans=s[i:j+1]
            i=piv+1
            j=piv
            cnt=0
            while i>0 and j<n-1:
                if s[i-1]==s[j+1]:
                    cnt+=2
                    i-=1
                    j+=1
                else:
                    break
            if L<cnt:
                L=cnt
                Ans=s[i:j+1]
        return Ans
```

代码运行截图 (至少包含有"Accepted")

通过



melocotón 提交于 2024.11.30 14:27

洛谷

12029: 水淹七军

bfs, dfs, <http://cs101.openjudge.cn/practice/12029/>

思路:

审题和解题过程得到群聊老师同学的帮助:

- 1) 从一个点扩散的水面高度相同;
- 2) 和经典的洪水填充不同, 一个点当淹过它的水面高度更高时, 需要反复入队出队。

通过对样例的理解, 淹没 (I, J) 是指最终 (I, J) 处的水面高度**严格大于** $H[I][J]$ 。

代码:

```
#include<bits/stdc++.h>
using namespace std;
const int dx[4]={1,0,-1,0},dy[4]={0,1,0,-1};
int m,n,x,y,h[203][203],w[203][203],in[203][203];
queue<pair<int,int> > q;
void up(int x,int y){if (!in[x][y]) in[x][y]=1,q.push(make_pair(x,y));}
int main()
{
    int p,K,x,y;
    scanf("%d",&K);
    while(K--){
        memset(h,0x7f,sizeof(h));
        memset(w,0xff,sizeof(w));
        memset(in,0,sizeof(in));
        while(!q.empty()) q.pop();
        scanf("%d%d",&m,&n);
        for(int i=1;i<=m;++i)
            for(int j=1;j<=n;++j)
                scanf("%d",&h[i][j]);
        scanf("%d%d%d",&x,&y,&p);
        for(int i=0;i<p;++i)
        {
            scanf("%d%d",&x,&y);
            w[x][y]=h[x][y];in[x][y]=1;
            q.push(make_pair(x,y));
        }
        while(!q.empty())
        {
            x=q.front().first;
            y=q.front().second;
            q.pop();in[x][y]=0;
```

```

        if (x==X&&y==Y&&w[X][Y]>h[X][Y]) break;
        for(int i=0;i<4;++i)
        {
            int u=x+dx[i],v=y+dy[i];
            if (u<1||u>m||v<1||v>n) continue;
            if (h[u][v]<=w[x][y]){
                if (w[u][v]<0) w[u][v]=w[x][y],up(u,v);
                else {if (w[u][v]<w[x][y])
                    w[u][v]=w[x][y],up(u,v);}
            }
        }
    }
    if (w[X][Y]>h[X][Y]) printf("Yes\n");
    else printf("No\n");
}
return 0;
}

```

代码运行截图 (至少包含有"Accepted")

12029: 水淹七军

Accepted

800kB

16ms

1105 B

G++

刚刚

02802: 小游戏

bfs, <http://cs101.openjudge.cn/practice/02802/>

思路:

不断误读题意。1) 地图“宽和高”坐标表示与习惯相反。2) “线段数”不是最短路径长度。3) 每组数据后输出一个空行，否则会Presentation Error。

有空格读入对C++不友好，需要用 `getline(cin, str)` 读入一个 `string` 类型，且在前面要再用一个 `getline` 消除上一行行尾。

类似最短路算法，但是“距离数组”还需要一维：线段进入这一格的方向。走入下一格，若方向与走入当前格相同，两个状态之间的“边权”为0，方向不同则为1。每个坐标有四个方向的状态节点，每个节点与至多四联通的四个方向，共16个节点有边，重复计数边除以2，故图的规模大约应是 $V \approx 4wh$, $E \approx 8V$ 。代码类似没有优先队列优化的Bellman-Ford，时间复杂度 $O(VE) = O(w^2h^2)$ 。

代码:

```

#include<bits/stdc++.h>
using namespace std;
const int dx[4]={1,0,-1,0},dy[4]={0,1,0,-1};
int n,m,s[79][79],d[79][79][4],in[79][79];
struct node{int x,y,a;};
queue<node> q;
int bfs(int e,int t,int u,int v)
{
    node o,j;
    int x,y,w,z,A,add;
    memset(d,0x7f,sizeof(d));
    memset(in,0,sizeof(in));
    while(!q.empty()) q.pop();
}

```

```

in[e][t]=1;
o.x=e,o.y=t,o.a=-1;
q.push(o);
while(!q.empty())
{
    o=q.front();
    x=o.x,y=o.y,A=o.a;
    q.pop();in[x][y]=0;
    for(int i=0;i<4;++i)
    {
        w=x+dx[i],z=y+dy[i];
        if (w<0||w>m+1||z<0||z>n+1) continue;
        if (w>0&&w<=m&&z>0&&z<=n&&s[w][z])
            if (w!=u||z!=v) continue;
        if (A== -1)
        {
            d[w][z][i]=1;
            if (!in[w][z])
            {
                j.x=w,j.y=z,j.a=i;
                q.push(j),in[w][z]=1;
            }
        }
        else {if (A==i) add=0;
              else add=1;
              if (d[w][z][i]>d[x][y][A]+add)
              {
                  d[w][z][i]=d[x][y][A]+add;
                  if (!in[w][z])
                  {
                      j.x=w,j.y=z,j.a=i;
                      q.push(j),in[w][z]=1;
                  }
              }
        }
    }
}
add=0x7f7f7f7f;
for(int i=0;i<4;++i)
    add=min(add,d[u][v][i]);
if (add==0x7f7f7f7f) return 0;
else return add;
}
int main()
{
    int t=0,tt,a,b,c,d,k;
    string S;
    scanf("%d%d",&n,&m);
    while(n!=0){
        memset(s,0,sizeof(s));
        getline(cin,S);
        for(int i=1;i<=m;++i)
        {
            getline(cin,S);
            for(int j=0;j<S.length();++j)
                if (S[j]=='x') s[i][j+1]=1;
        }
    }
}

```

```

printf("Board #d:\n", ++t); tt=0;
scanf("%d%d%d%d", &a, &b, &c, &d);
while(a!=0){
    printf("Pair %d: ", ++tt);
    k=bfs(b, a, d, c);
    if (k) printf("%d segments.\n", k);
    else printf("impossible.\n");
    scanf("%d%d%d%d", &a, &b, &c, &d);
}
printf("\n");
scanf("%d%d", &n, &m);
}
return 0;
}

```

代码运行截图 (至少包含有"Accepted")

02802: 小游戏

Accepted

408kB

16ms

1593 B

G++

刚刚

2. 学习总结和收获

如果作业题目简单，有否额外练习题目，比如：OJ“计概2024fall每日选做”、CF、LeetCode、洛谷等网站题目。

大搜索也是较为困难的题目，后两题尤其是“小游戏”调试时间很长。搜索题的任务是多样化的，不能一味套模板，要重新读题。分清不同的状态表示方式。